

Big MFA — Android

Rapport d'audit et de durcissement sécurité

Date du rapport : 27/09/2026

Application : com.bigfortravel.bigmfa (TOTP/HOTP, GPLv3)

Éditeur : BigForTravel — Patrick DJEUZONG

Dépôt : github.com/bigfortravel/big-mfa

Outil / méthode	Résultat
Audit manuel du code (sécurité)	3 lacunes identifiées et corrigées
detekt (analyse statique Kotlin)	295 → 294 constats, 1 anomalie de structure corrigée
Vérification CVE (équivalent manuel OWASP Dependency-Check)	Bouncy Castle mis à jour 1.79 → 1.85
SonarQube Community (local)	56 → 52 problèmes, Sécurité : 9 → 7 problèmes ouverts

Ce document consolide l'ensemble des vérifications de sécurité et de qualité menées sur l'application Big MFA (version Android native), les anomalies identifiées, les corrections apportées, et les points restants documentés comme acceptés ou non applicables.

1. Périmètre et méthodologie

L'audit a porté sur l'intégralité du code source de l'application Android Big MFA (Kotlin, Jetpack Compose), incluant le cœur cryptographique, la gestion du Keystore, le service de saisie automatique (Autofill), la gestion du coffre chiffré, et la configuration du projet (manifeste, Gradle).

Quatre méthodes complémentaires ont été utilisées :

- **Revue manuelle du code** — lecture directe des fichiers critiques (cryptographie, Keystore, verrouillage, Autofill) pour identifier les écarts entre le comportement annoncé à l'utilisateur et le comportement réel du code.
- **detekt** — analyse statique Kotlin (équivalent d'ESLint pour ce langage), exécutée en ligne de commande sur l'ensemble du code source principal.
- **Vérification manuelle des CVE** — recherche des vulnérabilités connues publiées pour chaque version exacte des bibliothèques tierces utilisées (Bouncy Castle, CameraX, ML Kit).
- **SonarQube Community Edition** (instance locale) — analyse Gradle couvrant fiabilité, sécurité, maintenabilité et duplication de code.

2. Audit manuel du code — 3 lacunes corrigées

Constat	Risque	Correction apportée
Aucun FLAG_SECURE sur l'activité principale	Capture d'écran et enregistrement possibles pendant l'affichage d'un code TOTP ; la vignette du sélecteur d'apps récentes exposait le contenu réel de l'écran.	Ajout de WindowManager.LayoutParams.FLAG_SECURE dans MainActivity.onCreate().
Compteur anti-bruteforce uniquement en mémoire	Un attaquant avec le téléphone en main pouvait forcer la fermeture du processus entre chaque lot de tentatives pour réinitialiser le compteur de blocage.	Persistance chiffrée du compteur (clé Keystore dédiée non-biométrique) via un nouveau composant BruteForceStateStore.
AutoLockManager jamais appelé dans l'application	Le verrouillage automatique après inactivité (annoncé « 2 min » dans Paramètres) n'était en réalité jamais déclenché — code écrit mais non branché.	Intégration réelle via ProcessLifecycleOwner : reverrouillage effectif au retour au premier plan après le délai configuré.

Les trois corrections ont été testées manuellement sur appareil et confirmées fonctionnelles.

3. Analyse statique — detekt

295 constats initiaux sur l'ensemble du code source principal (hors tests unitaires). Répartition par catégorie :

Règle	Nombre	Nature
MagicNumber	104	Style — nombres non extraits en constantes nommées
NewLineAtEndOfFile	62	Style — absence de ligne vide finale
FunctionNaming	46	Faux positifs — convention PascalCase attendue pour les fonctions @Composable
MaxLineLength	34	Style — lignes visuellement longues
LongMethod	24	Qualité — fonctions longues, courant en UI Compose
ReturnCount	7	Qualité — retours multiples, souvent des gardes de sécurité légitimes
LongParameterList	5	Qualité — fonctions à nombreux paramètres
CyclomaticComplexMethod	3	Qualité — complexité logique élevée (Autofill notamment)
Autres (ThrowsCount, MatchingDeclarationName, etc.)	9	Style mineur
InvalidPackageDeclaration	1	Anomalie réelle — corrigée (voir ci-dessous)

Anomalie corrigée

Le fichier `CreateVaultScreen.kt` était physiquement situé dans `ui/` tout en déclarant le package `com.bigfortravel.bigmfa.ui.setup` — incohérence de rangement, sans impact fonctionnel (Kotlin l'accepte), mais source de confusion en maintenance. Le fichier a été déplacé vers `ui/setup/` via le refactoring automatique d'Android Studio. Nouvelle analyse confirmée : 295 → 294 constats, aucune régression.

4. Vérification des vulnérabilités connues (équivalent manuel d'OWASP Dependency-Check)

L'outil OWASP Dependency-Check nécessite un accès à la base de données NVD (National Vulnerability Database) en ligne, indisponible dans l'environnement d'exécution utilisé pour cet audit. Une recherche manuelle des CVE publiées a été menée pour chaque bibliothèque et chaque version exacte utilisée par le projet, atteignant le même objectif de couverture.

Bibliothèque	Version auditée	CVE trouvées	Verdict
Bouncy Castle (bcprov-jdk18on)	1.79	CVE-2026-0636 CVE-2026-5598	Mise à jour → 1.85

Bibliothèque	Version auditée	CVE trouvées	Verdict
CameraX (core/camera2/lifecycle/view)	1.6.0	Aucune	Conforme
ML Kit barcode-scanning	17.3.0	Aucune	Conforme

Détail des CVE Bouncy Castle 1.79

- **CVE-2026-0636** (injection LDAP, module LDAPStoreHelper) — concerne la validation de certificats X.509 via annuaire LDAP. Big MFA n'utilise ni LDAP ni validation de certificat : **aucun impact réel**.
- **CVE-2026-5598** (canal de temporisation, module FrodoEngine) — concerne l'algorithme post-quantique FrodoKEM. Big MFA utilise exclusivement AES-256-GCM-SIV et Argon2id : **aucun impact réel**.

Mise à jour effectuée par précaution (hygiène de dépendances, éviter un signalement automatique — GitHub Dependabot — sans contexte sur le dépôt public) plutôt que par nécessité de sécurité immédiate.

5. Analyse SonarQube Community (instance locale)

Comparatif avant / après corrections

Indicateur	Avant	Après
Total des problèmes	56	52
Sécurité — note globale	D	D*
Sécurité — problèmes ouverts	9	7
Fiabilité	A (0 problème)	A (0 problème)
Maintenabilité	A (47 problèmes)	A (45 problèmes)
Duplication de code	0.8 %	0.8 %
Quality Gate	Passed	Passed

* Note inchangée car deux signalements restants (détaillés ci-dessous) sont documentés comme non applicables — la note SonarQube ne descend qu'au retrait effectif de tous les signalements « High », y compris les faux positifs.

Corrections apportées suite à l'analyse

Signalement	Fichier	Correction
Obfuscation désactivée en release	app/build.gradle.kts	optimization.enable = false → true
Accès stockage externe (export de sauvegarde)	SettingsScreen.kt	getExternalFilesDir() → cacheDir (privé à l'app)
Accès stockage externe (export QR, ZIP)	ExportQrScreen.kt	getExternalFilesDir() → cacheDir (privé à l'app)
Sauvegarde cloud / transfert non maîtrisée	data_extraction_rules.xml backup_rules.xml	Exclusion explicite de vault.json et bigmfa_security_state

Points restants — documentés comme non applicables

Signalement	Analyse
Obfuscation toujours signalée (build.gradle.kts)	Faux négatif de l'outil : la nouvelle syntaxe Gradle (optimization { enable = true }) n'est probablement pas encore reconnue par le moteur de règles SonarQube. Configuration réelle vérifiée correcte (build release fonctionnel avec obfuscation active).
Sauvegarde — manifeste debug (app/src/debug/AndroidManifest.xml)	Ce fichier ne définit aucune règle de sauvegarde propre ; Android fusionne automatiquement ses réglages avec le manifeste principal (déjà corrigé) au moment de la compilation. L'outil semble analyser le fragment isolé, avant fusion — comportement final du build de debug déjà protégé par héritage.

Signalement	Analyse
usesCleartextTraffic implicite (2 manifestes)	Impact réel nul : Big MFA n'effectue aucune requête réseau directe (liens de don/GitHub ouverts dans le navigateur externe, hors du processus de l'app).
Clés Keystore sans authentification biométrique	Choix de conception volontaire : ces clés ne protègent jamais les secrets TOTP du coffre (uniquement le compteur anti-bruteforce et un test matériel), seule la vraie clé biométrique protège les données sensibles.
verification-metadata.xml absent	Fonctionnalité Gradle avancée (vérification cryptographique des artefacts téléchargés) — effort jugé disproportionné au risque réel pour ce projet.

6. Synthèse et recommandations

L'ensemble des lacunes de sécurité à impact réel identifiées durant cet audit a été corrigé et validé par test manuel sur appareil. Les points restants ouverts dans SonarQube sont documentés individuellement : limites connues de l'outil d'analyse, ou risques déjà maîtrisés par la conception du logiciel.

Statut final par domaine

Domaine	Statut
Cryptographie du coffre (AES-256-GCM-SIV, Argon2id)	Conforme
Protection biométrique (Keystore, invalidation, StrongBox/TEE)	Conforme
Protection anti-capture d'écran	Corrigé
Protection anti-force brute (persistance)	Corrigé
Verrouillage automatique après inactivité	Corrigé
Confidentialité des exports (stockage)	Corrigé
Exclusion du coffre des sauvegardes système	Corrigé
Obfuscation du binaire de production	Corrigé
Dépendances tierces à jour	À jour
Service Autofill (biométrie stricte, vérification de domaine)	Conforme

Recommandations pour la suite

- Relancer une analyse SonarQube et detekt après tout ajout de fonctionnalité significatif, avant publication sur le Play Store.
- Surveiller les futures alertes GitHub Dependabot une fois le dépôt public, et documenter chaque alerte comme applicable ou non, à l'image de ce rapport.
- Envisager l'ajout de tests de couverture (actuellement 0 % rapporté, bien que des tests unitaires existent — configuration JaCoCo à intégrer à l'analyse).